

Array Operations: Where the Real Cost Hides

Programming and Data Structures — Week 2, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state the time complexity of inserting and deleting at the end, the middle, and the start of an array, and explain precisely why each of these three positions costs differently; trace, element by element, the shifting of memory that a middle or start insertion or deletion actually requires; compare Python's `list.insert(0, x)` against `list.append(x)` using real measured numbers rather than only asymptotic notation; and choose, for a given program's access pattern, which end of a list to grow from to avoid unnecessary cost.

Outline

Today covers a cinema-queue analogy for why position matters when inserting; the easy case of inserting or deleting at the end of an array; the shifting problem created by inserting or deleting in the middle, traced element by element; the worst case of inserting or deleting at the very start, also traced fully; a return to searching, connecting linear and binary search from Lecture 1 to the cost of keeping a list sorted after insertions; real measurements of these costs in Python; the MTech-track material on why `collections.deque` exists as a second, complementary structure; an in-class quiz; and a summary.

The cinema-queue analogy

Picture 200 people queuing for a cinema, standing shoulder to shoulder with no gaps between them — this is the array from Lecture 1, made physical. Someone joining at the very back of the queue is trivial: they simply stand behind the last person, and nobody else needs to move. Someone insisting on joining partway into the queue forces every single person standing behind that point to shuffle back one space, to open a gap. Someone insisting on joining at the very front forces the *entire* queue of 200 people to shuffle back one space each. The physical effort required in each case — none, some, or all of the queue moving — is exactly the computational cost this lecture formalizes for array insertion and deletion.

Insert/delete at the end: the easy case

```
scores = [67, 88, 45, 92, 71]
scores.append(60)      # [67, 88, 45, 92, 71, 60]
scores.pop()          # removes 60, back to [67, 88, 45, 92, 71]
```

Appending to or popping from the end of an array requires moving no existing elements at all — the new element occupies the next free slot, or the last element is simply marked as no longer present. The cost is $O(1)$ for both operations, setting aside the occasional resize covered in Lectures 2 and 3, which amortized analysis already showed does not change this $O(1)$ average. This is the cinema-queue “joining at the back” case exactly.

Insert in the middle: setting up the trace

```
scores = [67, 88, 45, 92, 71]
scores.insert(2, 99)    # insert 99 at index 2
```

Consider inserting the value 99 at index 2 of a 5-element array. Before the insertion, the array holds 67, 88, 45, 92, 71 at indices 0 through 4. After a correct insertion, it must hold 67, 88, 99, 45, 92, 71, with 99 now at index 2 and every element that used to be at index 2 or later shifted one position to the right. The next slide traces exactly how this shift happens, one element at a time.

Insert in the middle: the trace, element by element

The shift must proceed from the highest index down to the insertion point, not the other way around: step 1 moves the element at index 4 (71) into the newly available index 5; step 2 moves index 3’s element (92) into index 4; step 3 moves index 2’s element (45) into index 3; and step 4 finally writes 99 into the now-vacated index 2. The order matters — shifting left to right instead (starting from index 2) would overwrite the value at index 3 with 45 before that index 3 value had been copied anywhere, permanently losing data. This right-to-left discipline is a detail every from-scratch array implementation must get right.

Insert in the middle: counting the cost

Inserting at index k into an array of n elements requires shifting exactly $n - k$ elements — every element from index k to the old last index moves one position to the right. Inserting near the end, where k is close to n , shifts very few elements and is cheap; inserting near the start, where k is close

to 0, shifts nearly all n elements. Averaged uniformly over all possible insertion positions, the expected number of shifts is roughly $n/2$, which is still $O(n)$ — linear in the size of the array, unlike the $O(1)$ end-insertion case.

Delete in the middle: the mirror image

```
scores = [67, 88, 99, 45, 92, 71]
del scores[2]      # remove index 2 (99)
```

Deletion is the mirror image of insertion, and the shift direction reverses accordingly. Removing the element at index 2 requires shifting every element after it one position to the *left*, closing the gap the removal created: the element at index 3 moves into index 2, index 4's element moves into index 3, index 5's moves into index 4, and the array's reported length shrinks by one. Just as with insertion, the direction is not arbitrary — shifting left to right here (starting from the end) would work too in this particular direction, but the general principle to internalize is that the direction must always avoid overwriting a value before it has been read.

Insert/delete at the start: the worst case

```
scores = [67, 88, 45, 92, 71]
scores.insert(0, 99)      # every single element must shift right
```

Inserting at index 0 is the special case $k = 0$ in the shift formula from two slides ago, giving $n - 0 = n$ — every single element in the array must shift, with no exceptions. This is the cinema-queue's "join at the very front" case: the entire line of 200 people moves. Deleting from index 0 is symmetric: every remaining element shifts left by one to close the gap at the front. Both operations cost $O(n)$, the worst position possible for either operation on a plain array.

Summary table: cost by position

This table is worth memorizing precisely, because it recurs in almost every later data structure decision in this course: inserting or deleting at the end of an array costs $O(1)$ (amortized, per Lectures 2 and 3), while inserting or deleting anywhere else costs $O(n)$ in the worst case, due to shifting. Accessing an element by index, by contrast, is always $O(1)$ regardless of position, since Lecture 1's address formula does not depend on where in the array the target index sits.

Reconnecting to search from Week 1

This connects directly back to Week 1's lecture on Big-O. Linear search costs $O(n)$, while binary search costs $O(\log n)$, but binary search requires the array to already be sorted. Keeping an array sorted as new elements arrive means every insertion must land at the *correct* sorted position, not merely be appended anywhere. Finding that correct position costs only $O(\log n)$, using binary search. But making room for the new element at that position still costs $O(n)$, from shifting. The net cost of inserting into a sorted array is therefore $O(n)$ overall, dominated entirely by the shifting step — the fast $O(\log n)$ search does not save the operation from being linear overall.

Measuring it for real in Python

```
import time

def time_it(fn):
    start = time.perf_counter()
    fn()
    return time.perf_counter() - start

n = 100_000
data = list(range(n))

t_end = time_it(lambda: data.append(0) or data.pop())
t_start = time_it(lambda: data.insert(0, 0) and data.pop(0))
print(f"end: {t_end:.6f}s    start: {t_start:.6f}s")
```

Running this comparison on a 100,000-element list should show the start-insertion timing measurably larger than the end-insertion timing, and critically, this gap should *widen* as n grows — doubling n should roughly double t_{start} while t_{end} stays essentially flat. This is the $O(n)$ versus $O(1)$ distinction made directly observable, not merely asserted from theory; running this same experiment yourself with a few different values of n before the lab is strongly encouraged.

MTech addition — why `collections.deque` exists

```
from collections import deque
d = deque([67, 88, 45, 92, 71])
d.appendleft(99)    #  $O(1)$  — NOT  $O(n)$  like list.insert(0, ...)
```

Python's `collections.deque` exists specifically to solve the $O(n)$ start-insertion problem this lecture just measured. Internally, a deque is implemented as a doubly linked list of fixed-size blocks rather than one contiguous array, which allows both `appendleft` and `append` to run in $O(1)$ — a genuine structural trade-off, not a trick: a deque gives up the $O(1)$ *random-access* indexing that a plain array provides, since finding an arbitrary middle element in a linked structure requires traversal (Week 4 covers this cost precisely). The practical rule: use `list` when a program mostly indexes by position and appends at the end; use `deque` when a program needs to push and pop from both ends efficiently, such as a sliding window or a breadth-first search queue (Week 12).

MTech addition — the trade-off table

This trade-off table summarizes the entire design decision: `list` wins on end-append and index access, both $O(1)$; `deque` wins on start-append, also $O(1)$, but pays for it with $O(n)$ index access, since reaching an arbitrary middle element requires walking the linked structure of blocks. Neither structure is strictly better — the correct choice depends entirely on which operations a specific program actually performs most often, which is a recurring theme for the rest of this course.

A full worked trace tying it all together

Tying together search and shifting in one complete trace: maintaining a sorted leaderboard of 6 scores and inserting a new score of 80. Step 1 uses binary search to find the correct sorted position, requiring roughly $\log_2 6 \approx 3$ comparisons — the fast part. Step 2 shifts the three elements from index 3 onward (88, 92, 99) one position to the right to open a gap, exactly as traced earlier in this lecture for the middle-insertion case. Step 3 writes 80 into the now-open index 3. The result is a correctly sorted 7-element leaderboard, at a total cost dominated by the $O(n)$ shift in step 2, not the $O(\log n)$ search in step 1 — precisely the conclusion reached earlier in this lecture, now traced completely on a concrete example.

Exercise

As an exercise, implement `insert_sorted(arr, x)` using only Python's plain `list`: locate the correct insertion position with binary search ($O(\log n)$), then insert there using `list.insert` ($O(n)$ for the shift). Time this function against a plain `arr.append(x)` call for $n = 10,000$ and again for $n = 100,000$, and confirm that the ratio between the two timings roughly doubles as n doubles — direct evidence that the operation's true cost is dominated by the $O(n)$ shift, not the $O(\log n)$ search, matching the analysis earlier in this lecture.

Quiz — predict the cost

For an array of 1,000 elements, rank these three operations from cheapest to most expensive before checking the answer: appending at the end; inserting at index 500, the middle; and inserting at index 0, the very start.

Quiz — answer

`arr.append(x)` is cheapest: $O(1)$ amortized, no shifting at all. `arr.insert(500, x)` is next: $O(n)$, but only about 500 elements actually shift, since it inserts roughly in the middle. `arr.insert(0, x)` is most expensive: still $O(n)$ asymptotically like the middle case, but the actual constant is worse — all 1,000 elements shift, roughly twice the work of the middle-insertion case, even though both are described by the same big- O class. This is a useful reminder that big- O hides constants that can still matter in practice.

Summary

Inserting or deleting at the end of an array costs $O(1)$; inserting or deleting in the middle costs $O(n)$, shifting roughly $n - k$ elements for insertion position k ; inserting or deleting at the very start costs $O(n)$ in the worst sense, shifting every element in the array. Index access remains $O(1)$ everywhere, regardless of position, since it is pure address arithmetic. `collections.deque` exists specifically to buy $O(1)$ insertion at both ends, at the cost of $O(1)$ index access degrading to $O(n)$. Next week's stacks and queues are, in essence, disciplined interfaces built directly on top of these exact trade-offs: a stack only ever touches one end (cheap by this lecture's analysis), and a queue's naive array implementation runs straight into the $O(n)$ start-deletion problem this lecture just measured — which is precisely why a real queue implementation needs more care than it first appears to.